



UNIVERSIDAD IBEROAMERICANA PUEBLA

Somnus Sleep Monitor

Sistema IoT para monitoreo inteligente de habitaciones

Reporte del Proyecto

Integrantes

Fernando R. M.
Arith Maldonado Arith
Ximena Guadalupe Verdi Toledo

Materia

Arquitecturas Programables Avanzadas

Profesor

Rafael Pérez Aguirre

Mayo 2026

Índice

1. Resumen	3
2. Introducción	3
3. Planteamiento del problema	3
4. Objetivo general	3
5. Objetivos específicos	3
6. Alcance del proyecto	4
7. Arquitectura general	4
8. Componentes utilizados	5
8.1. Hardware	5
8.2. Software	5
9. Pinout final del sistema	6
10.Desarrollo del prototipo físico	6
10.1. Vista general del prototipo	7
10.2. Implementación electrónica	7
10.3. Interior de la habitación	9
11.Dashboard local en Raspberry Pi	9
12.Aplicación desarrollada en FlutterFlow	10
12.1. Storyboard de la aplicación	11
12.2. Configuración y publicación	12
13.Integración con Firebase	13
14.Temperatura y clima en línea	13
15.Visión por cámara	13
16.Enlaces del proyecto	14
16.1. Aplicación FlutterFlow	14
16.2. Dominio publicado	14
16.3. Comando de exportación	14
17.Pruebas realizadas	14
18.Resultados obtenidos	14

19.Limitaciones	15
20.Mejoras futuras	15
21.Conclusión	15
22.Anexos de código	16
22.1. Archivo principal: dashboard.py	16
22.2. Control de hardware real: hardware_real.py	16
22.3. Conexión con Firebase: firestore_crud.py	16
22.4. Detector de visión con OpenCV	16
22.5. Detector de visión con MediaPipe	16
22.6. Pruebas individuales	17
22.7. Dependencias	17
23.Referencias	18

1. Resumen

El proyecto *Somnus Sleep Monitor* consiste en un prototipo funcional para monitorear condiciones básicas relacionadas con el descanso de una persona y controlar actuadores desde una Raspberry Pi 5.

El sistema integra sensores, actuadores físicos, una cámara USB, un dashboard local desarrollado en Python/PyQt5 y una aplicación de usuario creada en FlutterFlow. Además, se utiliza Firebase Firestore para guardar el estado actual, almacenar lecturas históricas y recibir comandos remotos desde la aplicación.

2. Introducción

El Internet de las Cosas permite conectar dispositivos físicos a internet para monitorear información y automatizar acciones. En este proyecto se aplicó este principio para construir una habitación inteligente capaz de detectar movimiento, controlar ventilación, accionar una cortina y visualizar información en tiempo real.

La Raspberry Pi 5 funciona como controlador principal, mientras que Firebase permite la comunicación entre el hardware y la aplicación desarrollada en FlutterFlow.

3. Planteamiento del problema

Durante el descanso, factores como temperatura, ventilación, movimiento, luz y posición de cortinas pueden afectar la comodidad del usuario. Por ello, se buscó crear un sistema capaz de monitorear el entorno y controlar actuadores físicos de forma local y remota.

El reto principal fue integrar hardware físico, software local, servicios en la nube y una aplicación visual dentro de una misma arquitectura funcional.

4. Objetivo general

Desarrollar un prototipo funcional de monitoreo y control para un ambiente de descanso utilizando Raspberry Pi 5, sensores, actuadores, dashboard local y sincronización con Firebase y FlutterFlow.

5. Objetivos específicos

- Leer movimiento mediante un sensor PIR HC-SR501.
- Encender un LED físico cuando se detecta movimiento.
- Controlar dos ventiladores mediante un módulo de relevadores.
- Controlar una cortina con un motor DC usando un puente H L298N.
- Consultar temperatura exterior en línea mediante Open-Meteo.

- Mostrar lecturas, gráficas, estados y controles en un dashboard local.
- Guardar estado actual e historial en Firebase Firestore.
- Crear una aplicación de usuario en FlutterFlow.
- Integrar una cámara USB para detección visual básica.

6. Alcance del proyecto

El proyecto llega a nivel de prototipo funcional. Se probó hardware real conectado a una Raspberry Pi 5, se enlazó con Firebase Firestore y se desarrolló una aplicación de usuario en FlutterFlow.

El sistema no pretende ser un dispositivo médico. La detección de sueño o estado del usuario se considera una estimación experimental basada en movimiento, cámara y condiciones del entorno.

7. Arquitectura general

La arquitectura del sistema se divide en cuatro capas principales:

- **Capa física:** Raspberry Pi 5, sensor PIR, LED, relevadores, ventiladores, puente H L298N, motor DC y cámara USB.
- **Capa de control local:** Python, PyQt5, gpiozero, lgpio, OpenCV y consulta de clima en línea.
- **Capa de nube:** Firebase Firestore para estado actual, historial y comandos remotos.
- **Capa de usuario:** aplicación FlutterFlow con login, dashboard, historial y configuración.

El flujo general de datos es:

Raspberry Pi → Sensores y cámara → Dashboard → Firestore → FlutterFlow

El flujo general de comandos es:

FlutterFlow → Firestore → Raspberry Pi → Actuadores

8. Componentes utilizados

8.1. Hardware

- Raspberry Pi 5.
- Sensor PIR HC-SR501.
- LED indicador de movimiento.
- Módulo de relevadores.
- Dos ventiladores.
- Puente H L298N.
- Motor DC para cortina.
- Cámara USB.
- Protoboard y cables jumper.
- Fuente externa para motores y ventiladores.

8.2. Software

- Python.
- PyQt5.
- pyqtgraph.
- gpiozero.
- lgpio.
- OpenCV.
- Firebase Admin SDK.
- Firebase Firestore.
- Firebase Authentication.
- FlutterFlow.
- Open-Meteo API.

9. Pinout final del sistema

Los números GPIO corresponden a la numeración BCM de la Raspberry Pi.

Elemento	GPIO	Pin físico
PIR HC-SR501 OUT	GPIO 24	Pin 18
LED de movimiento	GPIO 12	Pin 32
Relevador ventilador 1	GPIO 17	Pin 11
Relevador ventilador 2	GPIO 27	Pin 13
L298N IN1	GPIO 5	Pin 29
L298N IN2	GPIO 6	Pin 31
L298N ENA	GPIO 13	Pin 33

Cuadro 1: Pinout general del sistema.

10. Desarrollo del prototipo físico

El prototipo físico fue construido como una maqueta de habitación inteligente. En ella se integraron ventiladores, sensores, cámara, actuadores y cableado para simular un entorno de monitoreo de descanso.

10.1. Vista general del prototipo

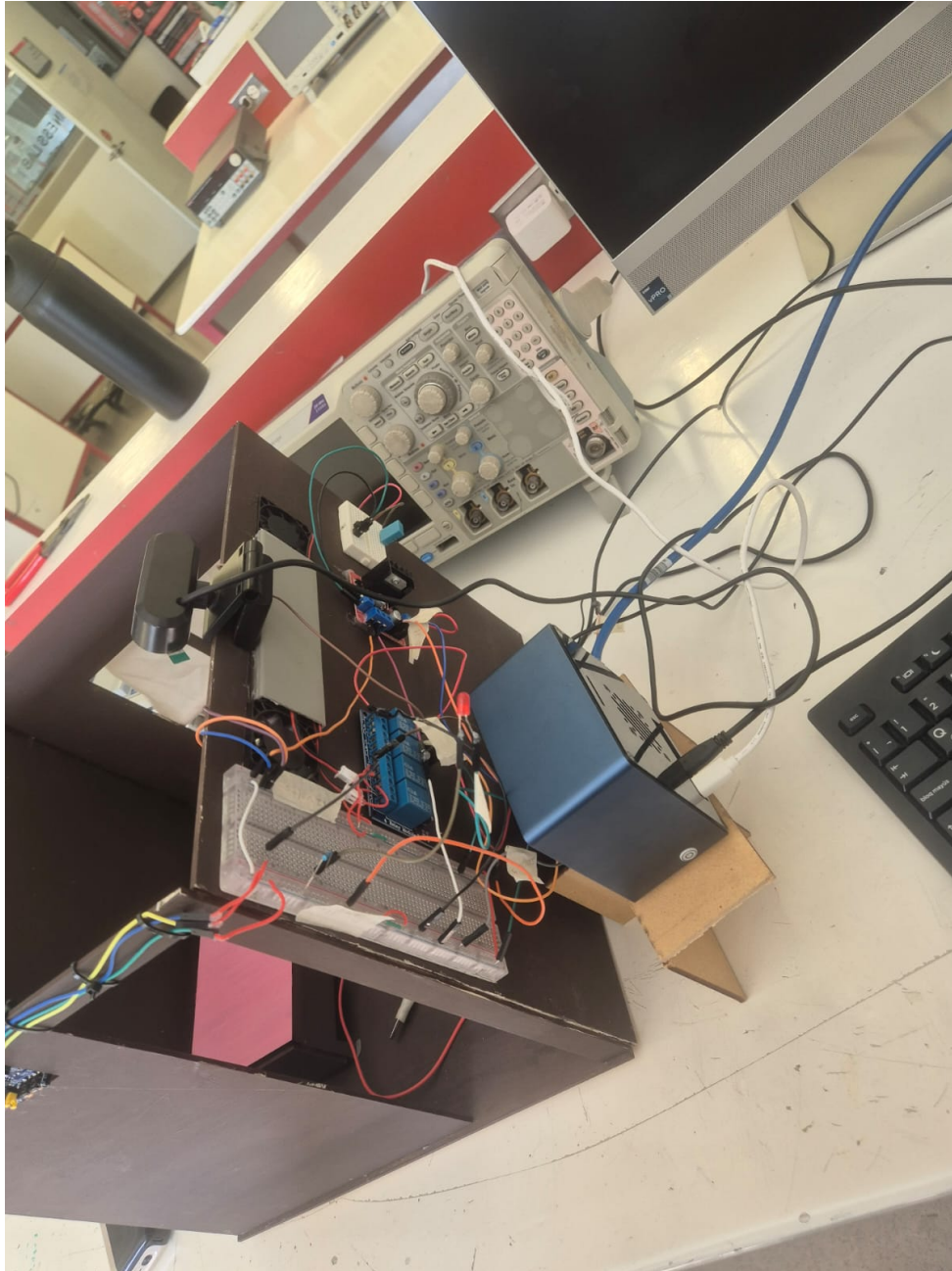


Figura 1: Vista general del prototipo físico de habitación inteligente.

10.2. Implementación electrónica

La implementación electrónica incluye relevadores, conexiones en protoboard, sensores, ventiladores y alimentación externa para actuadores. Esta sección corresponde únicamente al hardware físico del sistema.

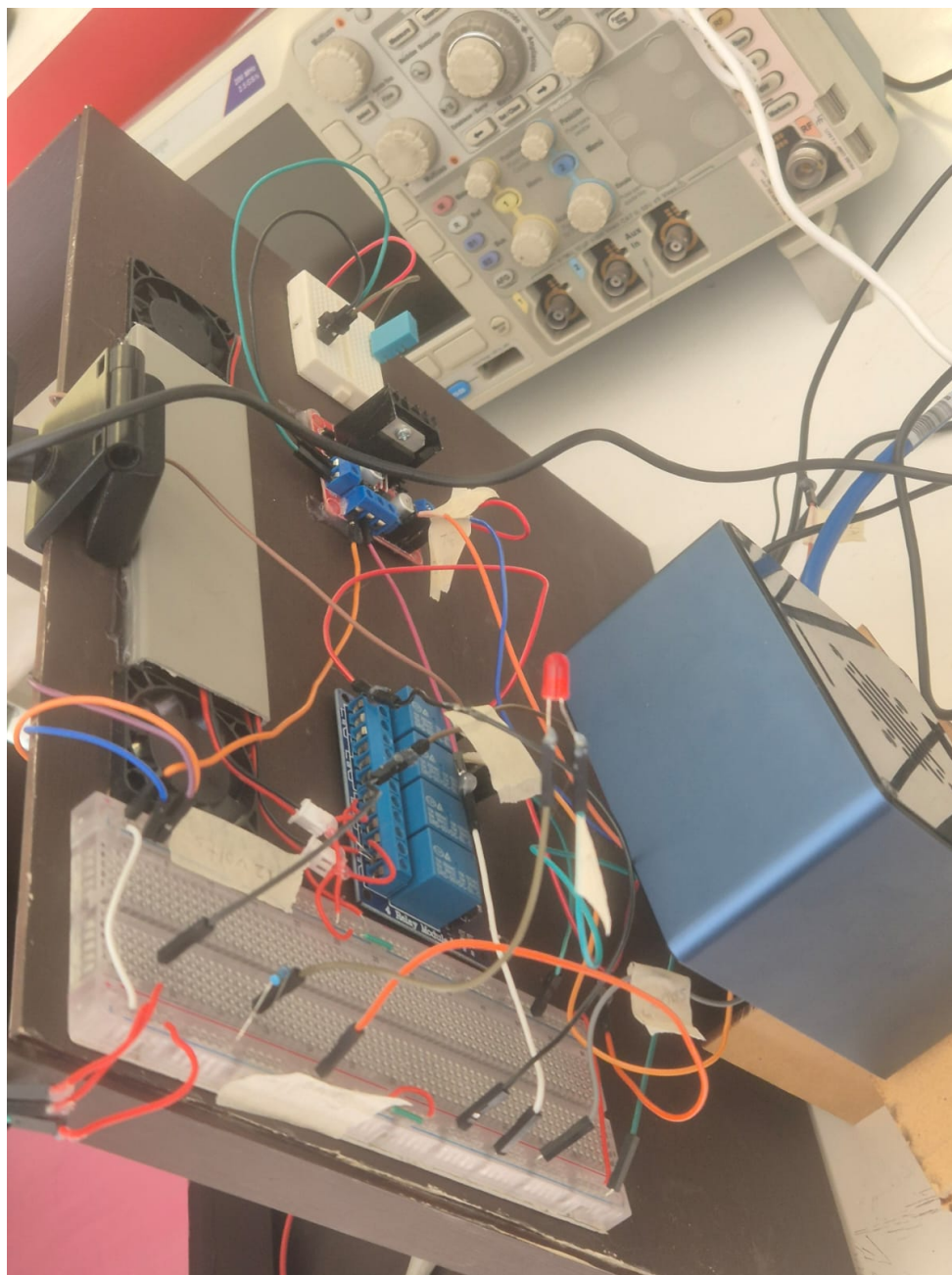


Figura 2: Implementación electrónica con relevadores, sensores y conexiones físicas.

10.3. Interior de la habitación



Figura 3: Interior del prototipo con ventiladores y cámara USB.

11. Dashboard local en Raspberry Pi

El dashboard local fue desarrollado en Python con PyQt5. Este permite visualizar temperatura, movimiento, actividad, estado de ventiladores, posición de cortina, estado de sesión y datos de visión por cámara.

El dashboard también permite controlar actuadores físicos desde la Raspberry Pi y sincronizar los datos con Firebase Firestore.

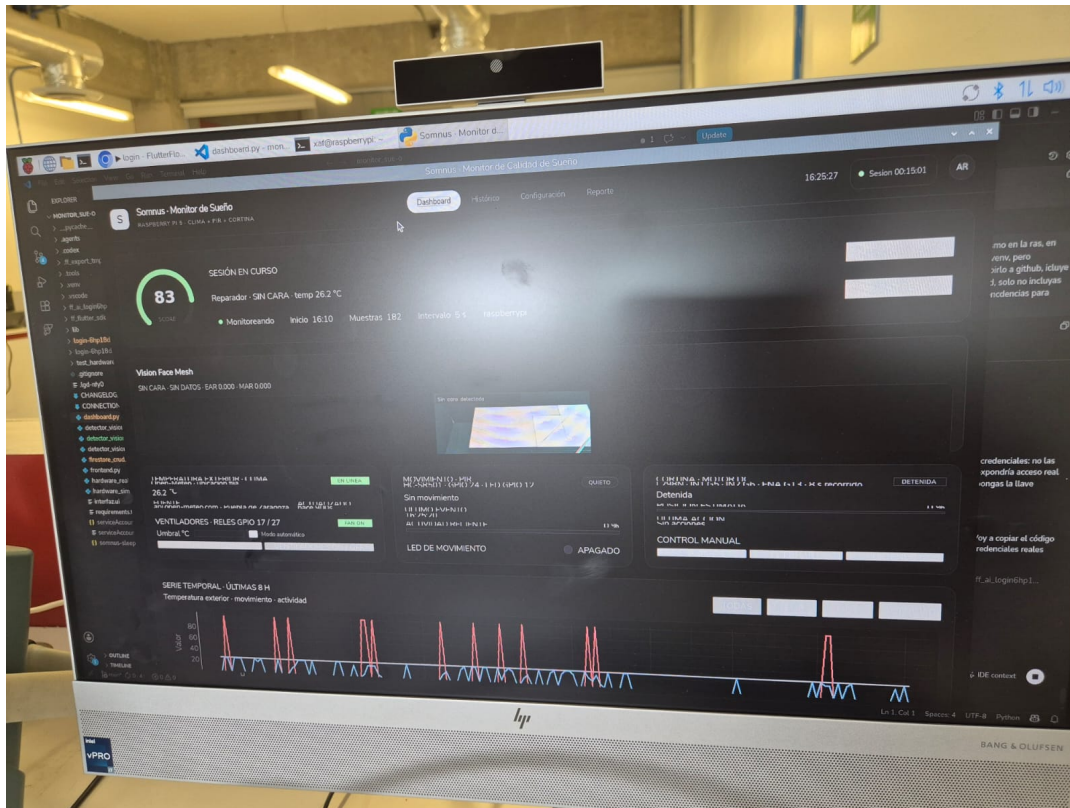


Figura 4: Dashboard local ejecutándose en la Raspberry Pi.

12. Aplicación desarrollada en FlutterFlow

La aplicación fue desarrollada en FlutterFlow y conectada a Firebase. Cuenta con páginas de login, pantalla principal, dashboard, historial y configuración.

Esta sección corresponde únicamente a la aplicación, no a los componentes físicos.

12.1. Storyboard de la aplicación

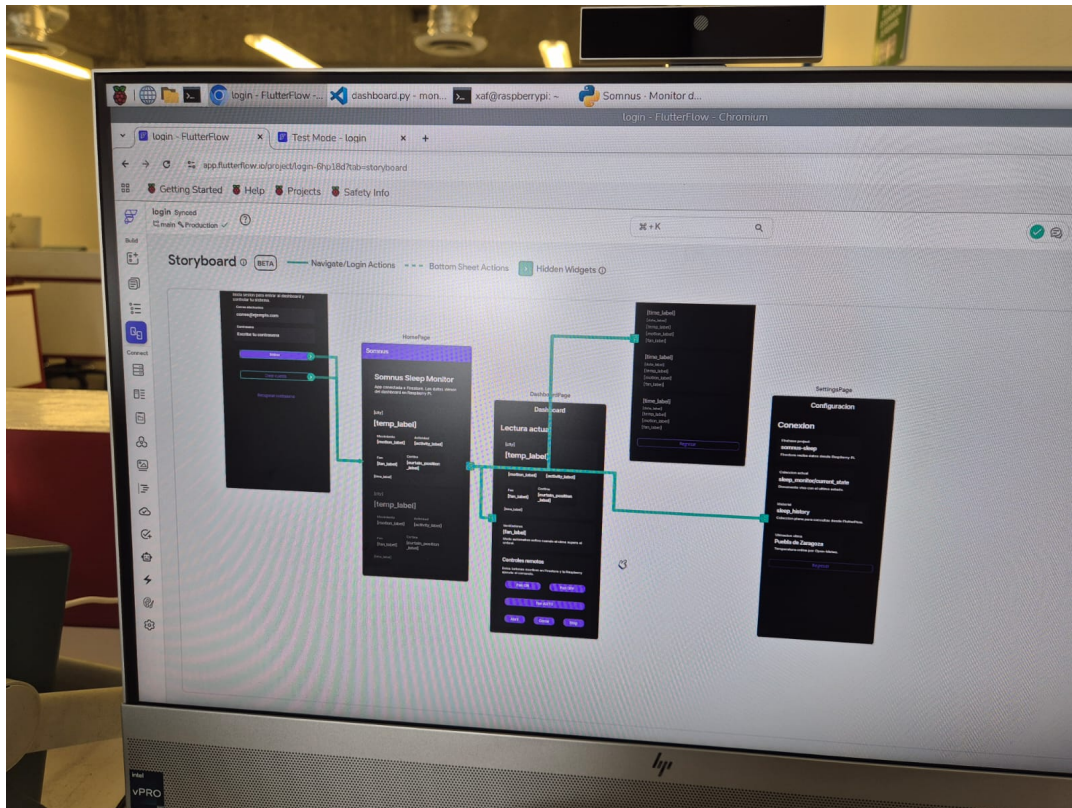


Figura 5: Storyboard de la aplicación desarrollada en FlutterFlow.

12.2. Configuración y publicación

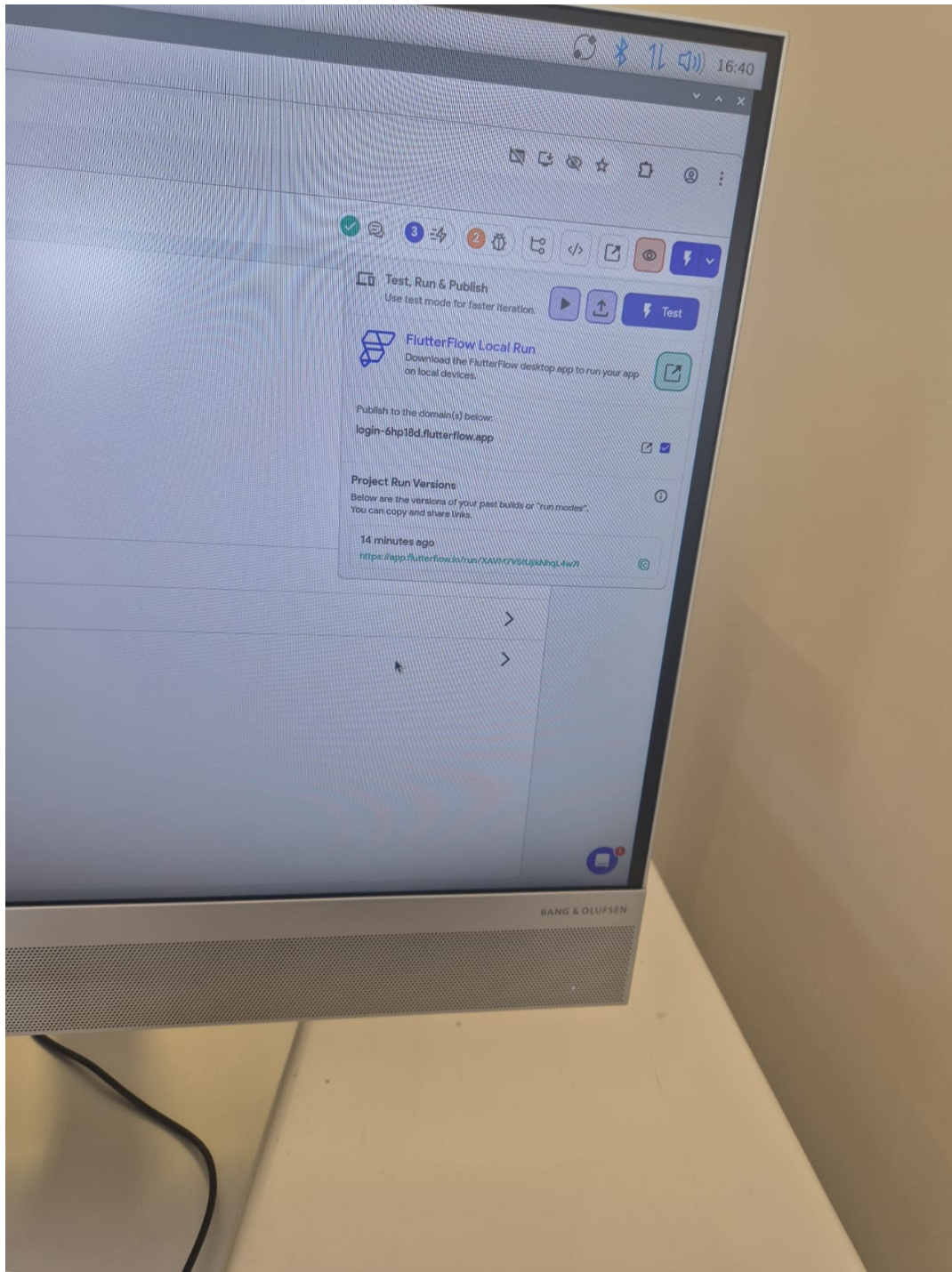


Figura 6: Configuración y publicación del proyecto en FlutterFlow.

13. Integración con Firebase

Firebase fue utilizado como base de datos en tiempo real para almacenar información del sistema y sincronizarla con FlutterFlow.

Las colecciones principales utilizadas son:

- **sleep_monitor/current_state:** documento con el estado actual del sistema.
- **sleep_history:** colección con lecturas históricas.
- **control_commands:** colección utilizada para enviar comandos desde FlutterFlow hacia Raspberry Pi.

Ejemplo conceptual de estado guardado:

```
1 {  
2   "temperature_c": 24.5,  
3   "city": "Puebla de Zaragoza",  
4   "pir_motion": true,  
5   "fan_on": true,  
6   "fan_auto": true,  
7   "curtain_position_percent": 100,  
8   "score": 82,  
9   "sleep_state_label": "DESPIERTO"  
10 }
```

14. Temperatura y clima en línea

Originalmente se intentó utilizar un sensor DHT11 para temperatura y humedad. Sin embargo, debido a lecturas inestables, se decidió retirar la medición física de temperatura en la versión final.

La temperatura se obtiene mediante Open-Meteo usando la ubicación de Puebla de Zaragoza. Esta lectura se utiliza para activar o desactivar ventiladores en modo automático.

15. Visión por cámara

El proyecto integra una cámara USB para estimar si hay rostro u ojos visibles. Inicialmente se planteó el uso de MediaPipe Face Mesh; sin embargo, debido a problemas de compatibilidad en Raspberry Pi, se agregó un respaldo mediante OpenCV y clasificadores Haar Cascade.

Esta función permite clasificar experimentalmente estados como despierto, dormido, sin cara o sin datos.

16. Enlaces del proyecto

16.1. Aplicación FlutterFlow

<https://app.flutterflow.io/run/XAVM7V5tUjIkNhqL4w71>

16.2. Dominio publicado

<https://login-6hp18d.flutterflow.app/>

16.3. Comando de exportación

```
1 flutterflow export-code --project login-6hp18d --project-environment  
  Production --include-assets --token 89df9a98-4fed-484d-a3da  
  -078414b95a43
```

Nota: por seguridad, los tokens y llaves privadas no deben publicarse en repositorios ni compartirse en documentos públicos.

17. Pruebas realizadas

- Prueba del sensor PIR.
- Prueba del LED indicador.
- Prueba de ventiladores mediante relevadores.
- Prueba de cortina con motor DC y puente H L298N.
- Prueba de cámara USB.
- Prueba de conexión con Firebase.
- Prueba de lectura y escritura desde FlutterFlow.

18. Resultados obtenidos

Se logró desarrollar un prototipo funcional capaz de monitorear movimiento, visualizar datos en un dashboard local, controlar ventiladores, accionar una cortina, guardar información en Firebase y permitir interacción desde una aplicación desarrollada en FlutterFlow.

Además, se logró integrar visión por cámara como prueba de concepto para estimar el estado del usuario.

19. Limitaciones

- El sistema no mide temperatura física interna de la habitación en la versión final.
- La temperatura en línea puede variar respecto al ambiente exacto del prototipo.
- La detección de sueño por cámara es experimental.
- La posición de la cortina se estima por tiempo y no por sensor físico.
- Los comandos remotos dependen de conexión a internet.

20. Mejoras futuras

- Agregar sensor de temperatura más confiable como DHT22, BME280 o SHT31.
- Agregar finales de carrera para la cortina.
- Implementar reglas de seguridad más estrictas en Firestore.
- Mejorar el modelo de visión por cámara.
- Crear un servicio permanente para iniciar el dashboard automáticamente.
- Mejorar el orden y fijación del cableado del prototipo.

21. Conclusión

Somnus Sleep Monitor demuestra la integración de hardware físico, software local y servicios en la nube mediante una Raspberry Pi 5. El proyecto permitió aplicar conceptos de arquitecturas programables, automatización, comunicación IoT, bases de datos en la nube e interfaces gráficas.

A pesar de los retos encontrados, como problemas con sensores, compatibilidad de librerías y sincronización entre dispositivos, el sistema logró funcionar como un prototipo completo de monitoreo y control inteligente de habitaciones.

22. Anexos de código

En esta sección se describen los archivos principales utilizados para el funcionamiento del sistema IoT desarrollado para el monitoreo inteligente de habitaciones.

22.1. Archivo principal: dashboard.py

Archivo encargado de ejecutar el dashboard local del sistema, permitiendo visualizar información de sensores, controlar actuadores y sincronizar datos provenientes de Firebase.

22.2. Control de hardware real: hardware_real.py

Este archivo administra la comunicación con los sensores y actuadores físicos conectados a la Raspberry Pi 5, incluyendo:

- Sensor PIR.
- LED indicador de movimiento.
- Ventiladores.
- Motor de cortina.
- Módulo de relevadores.

22.3. Conexión con Firebase: firestore_crud.py

Archivo encargado de establecer conexión con Firebase Firestore para:

- Lectura de datos.
- Escritura en tiempo real.
- Sincronización con FlutterFlow.
- Actualización remota del sistema.

22.4. Detector de visión con OpenCV

Archivo utilizado para pruebas de visión artificial mediante OpenCV, enfocado en la detección de rostro, ojos y presencia del usuario.

22.5. Detector de visión con MediaPipe

Implementación experimental utilizando MediaPipe para reconocimiento facial y posibles mejoras futuras en detección de movimiento y monitoreo de actividad.

22.6. Pruebas individuales

Se utilizaron archivos independientes para validar el funcionamiento de cada componente:

- `test_pir.py`: prueba del sensor PIR.
- `test_led.py`: prueba del LED indicador.
- `test_fans.py`: prueba de ventiladores.
- `test_curtain_motors.py`: prueba del motor de cortina.
- `test_dht11.py`: prueba histórica del sensor DHT11.

22.7. Dependencias

El sistema utiliza diversas librerías y dependencias para su funcionamiento:

- Python 3.
- Firebase Admin SDK.
- OpenCV.
- MediaPipe.
- PyQt5.
- `pyqtgraph`.
- `gpiozero`.
- `lgpio`.
- `openpyxl`.

Por motivos de seguridad y privacidad, no se anexan credenciales privadas, llaves JSON de Firebase ni configuraciones sensibles del sistema.

23. Referencias

- Firebase Documentation. Disponible en: <https://firebase.google.com/>
- FlutterFlow Documentation. Disponible en: <https://flutterflow.io/>
- Raspberry Pi Foundation. Disponible en: <https://www.raspberrypi.org/>
- OpenCV Documentation. Disponible en: <https://opencv.org/>
- MediaPipe Documentation. Disponible en: <https://developers.google.com/mediapipe>
- Python Documentation. Disponible en: <https://www.python.org/doc/>
- Open-Meteo Documentation. Disponible en: <https://open-meteo.com/>